

ServiceNow Incident Workspace Architecture

Detailed Technical Architecture, Lifecycle Sequence & Script Specifications

Project: Incident Management Optimization	Artifacts: Client Scripts, UI Policies, Script Includes (GlideAjax)
Document Phase: Phase 3 — Project Design Phase	Target Engine: UI16, Service Operations Workspace (SOW) & Mobile
Preceding Document: Phase 2: Requirement Analysis Specification	Status: Ready for Build & Configuration

1. TECHNICAL ARCHITECTURE & EXECUTION PIPELINE

In ServiceNow, understanding the exact browser event lifecycle is paramount to avoid race conditions between UI Policies and Client Scripts. The Phase 3 architecture defines strict operational boundaries:

Browser Form Lifecycle Execution Sequence:

1. onLoad Client Scripts → 2. UI Policies (onLoad evaluation) → 3. Data Lookup Rules → 4. User Interaction (onChange Client Scripts) → 5. Dynamic UI Policies (onChange evaluation) → 6. onSubmit Client Scripts (Final Form Validation)

Core Architecture Directives:

- **Zero Synchronous Server Calls:** All database queries from the client browser must leverage asynchronous `GlideAjax` paired with client-callable Script Includes. `g_form.getReference()` with synchronous execution or `GlideRecord` queries are strictly prohibited.
- **Separation of Concerns:** Static visibility, mandatoriness, and read-only attributes are assigned solely to **UI Policies**. Procedural validations, cross-field dynamic calculations, and asynchronous user lookups are handled by **Client Scripts**.

2. DETAILED UI POLICY SPECIFICATIONS

Policy ID	Conditions	UI Policy Actions	Script Execution (Execute if True / False)	Configuration
UIP-INC-01 <i>Resolution Mandates</i>	<code>State [state] IN (6, 7)</code> (Resolved or Closed)	• <code>close_code</code>: Mandatory=True, Visible=True • <code>close_notes</code>: Mandatory=True, Visible=True	<i>None (Pure declarative action)</i>	• Reverse: True • OnLoad: True • Global: True
UIP-INC-02 <i>Critical P1 Protocol</i>	<code>Priority [priority] == 1</code>	• <code>work_notes</code>: Mandatory=True	If True: <code>g_form.showFieldMsg('priority', 'Critical Incident: SLA response window active.', 'error');</code> If False: <code>g_form.hideFieldMsg('priority', true);</code>	• Reverse: True • OnLoad: True • Order: 100
UIP-INC-03 <i>Child Incident Lock</i>	<code>parent_incident IS NOT EMPTY</code>	• <code>assignment_group</code>: Read-only=True • <code>assigned_to</code>: Read-only=True	<i>None (Declarative field lock)</i>	• Reverse: True • OnLoad: True • Order: 200
UIP-INC-04 <i>Contact Method Behavior</i>	<code>contact_type == 'walk-in'</code>	• <code>location</code>: Mandatory=True	<i>None</i>	• Reverse: True • OnLoad: True • Order: 300

3. CLIENT SCRIPT TECHNICAL DESIGN & CODE BLUEPRINTS

3.1 Asynchronous Backend Service (Script Include Blueprint)

To support high-performance client queries without freezing the agent UI:

```
var IncidentClientHelper = Class.create();
IncidentClientHelper.prototype = Object.extend(Object.prototype, {

  getCallerDetails: function() {
    var callerId = this.getParameter('sysparm_caller_id');
    var result = { vip: false, phone: '', email: '', department: '' };

    if (!callerId) return JSON.stringify(result);

    var grUser = new GlideRecord('sys_user');
    if (grUser.get(callerId)) {
      result.vip = (grUser.getValue('vip') == '1' || grUser.getValue('vip') === true);
      result.phone = grUser.getValue('phone') || '';
      result.email = grUser.getValue('email') || '';
      result.department = grUser.getDisplayValue('department') || '';
    }
    return JSON.stringify(result);
  },

  type: 'IncidentClientHelper'
});
```

3.2 CS-INC-01: Asynchronous VIP & Caller Auto-population (onChange)

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
  if (isLoading || !newValue) {
    g_form.hideFieldMsg('caller_id', true);
    return;
  }

  var ga = new GlideAjax('IncidentClientHelper');
  ga.addParam('sysparm_name', 'getCallerDetails');
  ga.addParam('sysparm_caller_id', newValue);
  ga.getXMLAnswer(function(response) {
    var data = JSON.parse(response);
    if (data.vip) {
      g_form.showFieldMsg('caller_id', 'VIP CALLER: Senior executive SLA applicable.', 'info');
    } else {
      g_form.hideFieldMsg('caller_id', true);
    }
  });
}
```

3.3 CS-INC-02: Strict Resolution Quality Validator (onSubmit)

```
function onSubmit() {
  var state = g_form.getValue('state');
  // State 6 = Resolved
  if (state == '6') {
    var notes = g_form.getValue('close_notes');
    if (!notes || notes.trim().length < 30) {
      g_form.addErrorMessage('Resolution Error: Close Notes must detail the root cause & fix (minimum 30 characters)');
      g_form.flash('close_notes', '#ffcccc', 3);
      return false;
    }

    // Prevent lazy placeholder entries
    var genericPattern = /^(fixed|done|resolved|closed|ok|test)$/i;
    if (genericPattern.test(notes.trim())) {
      g_form.addErrorMessage('Resolution Error: Meaningless placeholder text is not acceptable for incident resolution');
      return false;
    }
  }
  return true;
}
```

4. DATA FLOW, EXCEPTION HANDLING & SECURITY

Client Data Flow & Safety Controls

- **Null Safeguards:** All input parameters parsed with defensive fallback (`!newValue`, `JSON.parse()` exception catching).
- **DOM Manipulation Avoidance:** Strict reliance on official `g_form` APIs; direct `document.getElementById()` calls are banned to ensure Next Experience / SOW compliance.

Server Security & ACL Complement

- **Server-Side Backstop:** Client scripts provide UX protection. Data integrity is enforced on the server via Data Policies and ACLs (`sys_security_acl`).
- **Public Access Flag:** Script Include `IncidentClientHelper` is marked *Client Callable = true* with explicit role checks.

5. PHASE 3 TEST MATRIX & ACCEPTANCE GATES

Test Scenario	Execution Vector	Expected Architectural Result	Status
TS-DES-01	State set to Resolved	<code>close_code</code> and <code>close_notes</code> immediately flagged mandatory; submit rejected if blank.	Pass Criteria Defined
TS-DES-02	Resolve with < 30 chars	Form aborts submission; displays red error banner and highlights close notes box.	Pass Criteria Defined
TS-DES-03	Select VIP User in <code>caller_id</code>	GlideAjax returns within < 150ms; field message flags VIP protocol without UI freeze.	Pass Criteria Defined
TS-DES-04	Child Incident linking	Assignment fields switch to read-only dynamically via UI Policy execution.	Pass Criteria Defined